

挑战项目一

基于特征工程的动漫人脸检测与关键点回归

一、项目背景

提到人脸检测，很多同学首先想到的是深度神经网络，似乎只有足够大的模型才能完成足够难的视觉任务。



但本项目要展示一个更有启发性的事实：**一张人脸并不一定要先被“深度理解”，也可以先被一组非常便宜、非常稳定的几何与灰度关系识别出来。**

参考工程 [cpu-face-hpc](#) 的核心经验是：在正面或近正面人脸检测中，**11 个由灰度、局部平滑和不同方向像素差构成的手工特征通道**，配合 **Depth-2 AdaBoost Cascade**，已经可以获得很好的人脸检测效果。

这些特征几乎只涉及整数加法、减法和除以 2（除以 2 可进一步变成位移运算）。虽然传统 Cascade 算法效果无法与现代 DNN 相比拟，但在低功耗设备甚至没有 FPU 的处理器（如 Cortex-M0/M3/M4）上仍有重要意义。

动漫人脸通常具有更清晰的轮廓、更强对比的眼睛和更规则的线条，因此一个自然的项目假设是：**如果连真实照片中的正面人脸都可以由这类简单特征识别，那么结构更鲜明、纹理更简洁的近正面动漫人脸，可能更容易被这类特征捕获。** 但动漫人脸也有夸张眼型、发型遮挡、侧脸、倾斜、画风差异和

对白框干扰，本项目要求亲手验证这些简单整数特征在**动漫图像**上有多强、会在哪些情况下失效、以及为什么失效。

本项目不是"调用模型 API"的项目，而是一个**从像素出发理解视觉系统**的项目。

二、项目目的

完成本项目后，应能够：

1. **理解 11 个手工特征通道**：从灰度图构建 11 个手工特征通道，并解释每个通道的含义；
2. **理解特征原理**：说明局部平滑为什么能抑制噪声，像素差为什么具有一定亮度鲁棒性；
3. **掌握弱分类器构造**：使用像素差构建 Depth-2 弱分类器（决策树）；
4. **掌握集成方法**：使用 AdaBoost 或等价集成方法组合弱分类器；
5. **理解 Cascade 加速原理**：构建多阶段 Cascade，并解释"早拒绝"为什么能节省计算；
6. **掌握迁移能力**：将真实人脸上的特征设计迁移到动漫人脸；
7. **掌握数据构建**：构建动漫人脸正负样本，并完成困难负样本挖掘；
8. **实现关键点回归**：实现 28 点动漫人脸形状回归；
9. **掌握评价方法**：使用 Precision、Recall、F1、IoU、NME 等指标评价系统；

三、项目概要

端到端流程：输入动漫图像 → 灰度化 → 图像金字塔 → 计算 11 个手工特征通道（原图 + 局部平滑 + 短/长距离方向梯度）→ 24×24 滑动窗口 → 像素差构成 Depth-2 决策树弱分类器 → AdaBoost 组合成强分类器 → 多级 Cascade 逐级"早拒绝"非人脸窗口 → 候选框 NMS/聚类合并 → 得到 Bounding Box → 人脸区域归一化 → 基于局部像素差的 28 点形状回归 → 结果可视化与导出。

核心思想：用最便宜的整数运算（加减、除以 2）提取稳定的灰度与几何关系，用大量"弱问题"按合理顺序组合成强判断，用 Cascade 把昂贵计算留给少数可疑窗口——从像素出发理解"简单特征如何组合成可用的人脸检测器"，并为 CPU 高性能优化（缓存、SIMD、定点化、多线程）铺路。

四、项目步骤

步骤一：环境搭建（uv 创建 Python 环境并安装依赖）

- 安装 uv
- 在项目目录创建虚拟环境并激活；
- 安装依赖并验证环境。

步骤二：复现并观察 11 通道特征

- 参考工程 [cpu-face-hpc](#)
- 实现 `compute_11_channels(gray)` 函数，输入 `uint8` 灰度图，输出 11 个 `uint8` 特征通道（原图灰度、 2×2 局部均值、跨 2 像素平滑、4 个短距离方向梯度、4 个长距离方向梯度）；
- 中间计算使用 `int16 / int32` 避免溢出，不使用现成卷积库直接生成全部结果；
- 选择至少 6 张图像（2 张真实正面人脸、2 张近正面动漫人脸、1 张带头发遮挡、1 张漫画复杂背景），绘制“原图 + 11 个通道”，观察哪些通道突出眼睛/轮廓、哪些易被头发或对白框干扰，并解释为什么后面 Cascade 能从这些通道中选到有效弱分类器。

步骤三：构建动漫人脸检测训练集

- **正样本**：从 Manga109 或 AnimeFace Dataset 数据裁剪人脸并缩放到 24×24 ，覆盖不同脸型、眼型、发型、轻微旋转、绘制风格和少量遮挡；
- **负样本**：必须来自真实动漫场景（背景、头发、衣服、手部、文字框、拟声词、建筑物等），而非随机噪声；
- **数据划分**：Train 75% / Validation 10% / Test 15%，必须避免数据泄漏（同一原图的不同裁剪不能跨训练/测试集，数据增强只作用于训练集），保存固定划分文件和随机种子。

步骤四：训练 Depth-2 AdaBoost Cascade

- 定义候选像素差特征（通道 `c` + 两个相对位置 `p1`、`p2`），一棵 Depth-2 弱树包含根节点特征与阈值、左右子节点特征与阈值、四个叶子输出分数；
- 采用随机候选 + 贪心搜索训练弱树，用 AdaBoost 更新样本权重，至少构建 **3 个 Stage** 组成 Cascade；
- 统计每个 Stage 的窗口保留情况，若第一级就拒绝大量真实人脸说明阈值过严，若最后仍有大量背景通过说明负样本不足；
- 完成**困难负样本挖掘**：用初始检测器扫描漫画页面，收集高置信度误检加入负样本重新训练，至少展示 6 个困难负样本并解释误检原因。

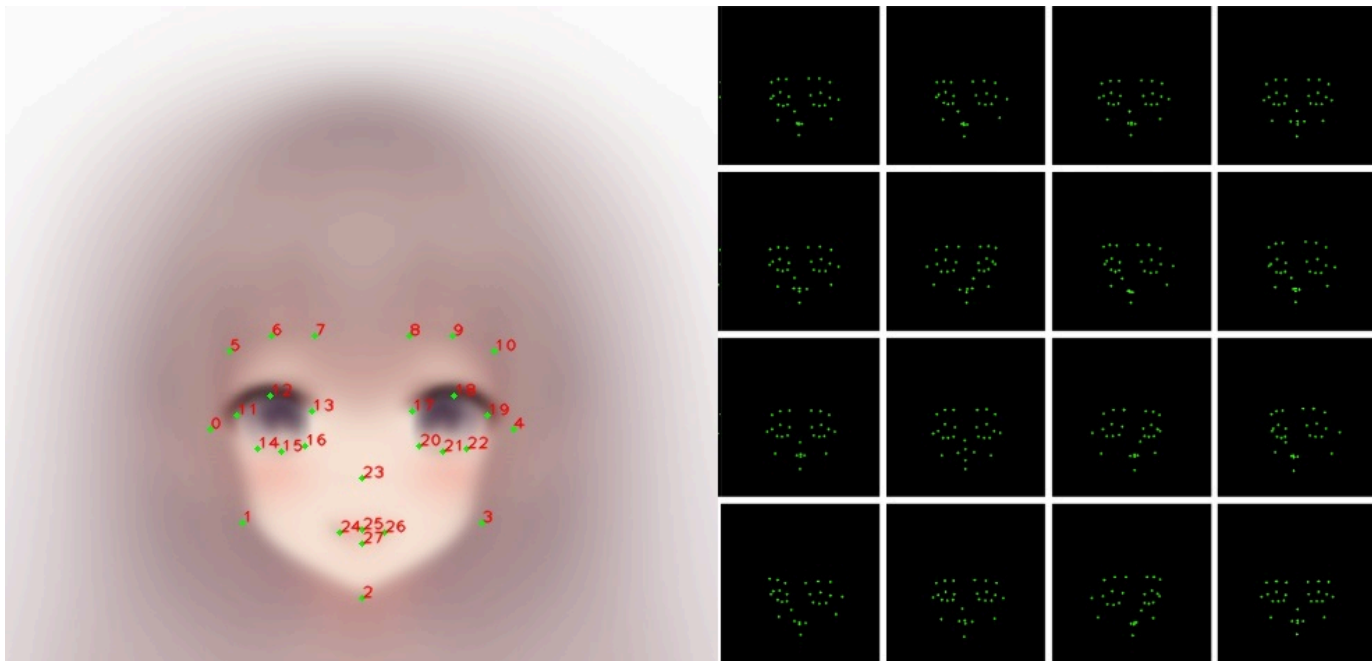
步骤五：多尺度搜索与候选框合并

- 构建图像金字塔（约 1.2 尺度增长，可对比 1.1 / 1.2 / 1.3），记录每层缩放尺寸、滑窗数量、通过 Cascade 的候选数、处理时间；
- 滑动窗口从 `step = 1` 逐像素扫描开始，再比较 `step = 2`，量化研究步长对结果的影响；
- 使用 NMS 或 Similar Rectangles 聚类合并同一张脸在相邻位置/尺度产生的多个候选框。

步骤六：28 点关键点回归

- 标注格式固定 28 个点，点顺序必须固定，被遮挡点使用 `visibility = 0`；
- 无现成 28 点数据集，可使用 [anime-face-detector](#) 预标注再人工修正，实现半自动化标注流水线；

- 基础方案：多级形状回归（初始平均形状 → 围绕关键点采样局部像素差 → Ridge Regression 预测 56 维形状残差 → 更新关键点，重复 3~5 轮）；
- 进阶方案：Fern / LBF 风格特征、可见点加权损失等。



步骤七：评价指标

- **检测**：IoU ≥ 0.5 判定匹配，报告 Precision、Recall、F1；
- **关键点**：NME（归一化平均误差，d 推荐使用双眼中心距离），只统计 visibility = 1 的点；
- **效率**：记录平均每张图检测时间、每个 Stage 通过率、不同步长窗口数量、整数版与浮点版输出是否一致。

步骤八：最终 Demo 与模型导出

- 提供命令行 Demo
(`python demo.py --image test.jpg --model-dir models/ --output results/test_vis.jpg`)，输出 Bounding Box、Detection Score、28 个关键点、结构化 JSON；
- 提供统一 Python 接口 `AnimeFaceDetector` (`detect()` 输入 OpenCV BGR 图像，返回原始图像坐标下的结果，按分数降序，检测不到返回空列表)；
- 必须保存：11 通道特征定义、Depth-2 弱树参数、Cascade Stage 阈值、图像金字塔参数、候选框合并参数、关键点回归模型、关键点编号与归一化配置、数据划分与随机种子。

四、提交内容

1. Python 源代码 (requirements.txt、README 运行说明、完整的代码)；
2. 构建的人脸 28 点关键点数据集 (图片以及标注)
3. 模型文件与配置文件；

4. 最终Demo（11个特征通道图、最终的检测效果图）；
5. 项目报告；

参考资料

1. [cpu-face-hpc](#)：11 通道手工特征、Depth-2 AdaBoost Cascade、CPU 高性能实现参考
2. [cpu-face-hpc / face_frontal.py](#)：教学版 Python 参考实现
3. [Manga109](#)：漫画图像与人脸框数据
4. [AnimeFace Dataset](#)：动漫人脸图像数据
5. [anime-face-detector](#)：动漫人脸 28 点关键点参考
6. Viola, P., & Jones, J. Rapid Object Detection using a Boosted Cascade of Simple Features